

Kubernetes Notes – Zero to Hero

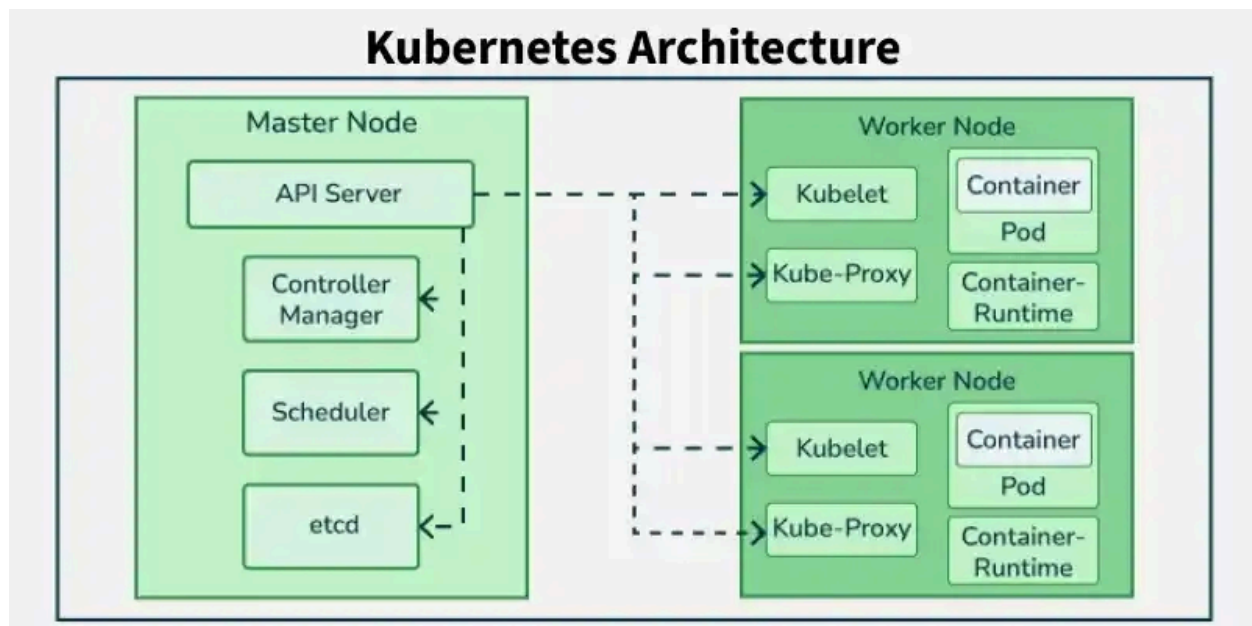
Why Kubernetes Over Docker?

Docker runs containers on a single machine. As applications scale, you need more than just containers — you need orchestration. Kubernetes provides four key advantages:

1. **Cluster** – runs across multiple machines by default
2. **Auto Healing** – restarts failed pods automatically
3. **Auto Scaling** – scales pods based on load
4. **Enterprise Support** – advanced load balancing, networking, security

Kubernetes Architecture

Kubernetes is split into two planes:



Control Plane (Master Node)

Manages the cluster. Contains:

Component	What It Does	Typical Failure Mode
etcd	Consistent, key-value store that holds the <i>desired</i> state of the	Corruption, loss of quorum, disk-full, stale

Component	What It Does	Typical Failure Mode
	entire cluster (objects, secrets, configuration).	snapshots.
API Server	Central HTTP(S) entry point for all client interactions (<code>kubectl</code> , controllers, UI). Validates, authorises, persists objects to etcd.	503 Service Unavailable, TLS handshake failures, high latency.
Scheduler	Reads pending Pods, consults node resource reports, applies policies (taints, affinity, priorities) → decides <i>where</i> a Pod should run.	Pods stuck in <code>Pending</code> because no node matches; scheduling loops.
Controller Manager	Runs a collection of <i>controllers</i> (Node, Replication, Deployment, StatefulSet, Service, etc.) that continuously reconcile <i>actual</i> state → <i>desired</i> state.	Controllers crash → no new Replicas, Service IP not allocated.
Cloud-Controller-Manager (optional)	Integrates with the underlying IaaS (AWS, GCP, Azure) – provisions LoadBalancers, Volumes, Node-labels, routes.	Cloud resources not created, mismatched tags.
Add-ons (CoreDNS, Metrics-Server, Ingress-Controller, Dashboard, etc.)	Extend cluster functionality; not part of the core control plane but are <i>de-facto</i> required for production.	DNS look-ups failing, metrics unavailable, ingress routes not created.

Data Plane (Worker Nodes)

Runs your applications. Each worker node has:

Component	What It Does	Why It's Needed
kubelet	Runs on every node, watches the API server for Pods assigned to that node, then uses the container runtime to start/stop	Guarantees the node's view of the world matches the control plane.

Component	What It Does	Why It's Needed
	containers, reports health/status.	
kube-proxy	Implements virtual IPs (ClusterIP) and load-balancing for Services using either iptables, IPVS, or userspace mode.	Makes a Service reachable from any pod without knowing the pod IPs.
Container Runtime	Executes OCI-compatible containers – Docker Engine, containerd, cri-o.	Actually runs the user workloads.
CNI Plugin (Container Network Interface)	Provides pod networking (assigns IP per pod, sets up overlay or native network, enforces network policies).	Enables <i>flat</i> pod-to-pod communication across nodes.
CRI-Shim (e.g., <code>dockershim</code> – now deprecated)	Bridge between kubelet (CRi) and a specific runtime.	Allows kubelet to be runtime-agnostic.
Node-level add-ons (e.g., <code>node-problem-detector</code> , <code>kube-let-metrics</code>)	Report local health – disk pressure, memory pressure, kernel errors.	Surface hardware / OS problems to the control plane.

Kubernetes vs Docker – Key Difference

Docker	Kubernetes
Runs on a single node	Cluster by default
No auto healing	Auto healing via ReplicaSet
Manual scaling	Auto scaling
Docker Engine + Docker Shim	Kubelet + any CRI runtime
Docker networking (bridge)	Kube-Proxy with iptables

Pod

A **pod** is the smallest deployable unit in Kubernetes. It is a wrapper around one or more containers.

- In Docker: `docker run -d -p 80:80 nginx`
- In Kubernetes: you define the same thing in a YAML file called `pod.yaml`

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - name: nginx
    image: nginx:1.14.2
    ports:
    - containerPort: 80
```

Apply it:

```
kubectl apply -f pod.yaml
kubectl get pods
kubectl get pods -o wide      # shows IP address
kubectl describe pod nginx    # full details, useful for debugging
kubectl logs nginx            # application logs
kubectl delete pod nginx
```

Why use Pods instead of containers directly?

- Declarative: all config is in YAML, not command-line flags
- Pods can hold multiple containers that share networking and storage (sidecar pattern)

- Kubernetes assigns an IP to the pod, not the container

Multi-container Pods

Containers inside the same pod:

- Share the same IP (talk via `localhost`)
- Share volumes
- Used for sidecar containers (e.g., service mesh, log shippers)

Deployment

A **Deployment** is the recommended way to run applications. It wraps pods and adds:

- **Auto Healing** via ReplicaSet
- **Scaling**
- **Zero-downtime upgrades**

How it works

`Deployment` → creates `ReplicaSet` → creates `Pods`

The ReplicaSet is a Kubernetes controller. It constantly watches: *"Is the actual number of pods equal to the desired count?"* If not, it creates or deletes pods.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.14.2
```

```
ports:
- containerPort: 80
```

bash

```
kubectl apply -f deployment.yaml
kubectl get deploy
kubectl get rs          # replica sets
kubectl get pods
kubectl delete pod <pod-name> # ReplicaSet will recreate it immediately
```

Pod vs Container vs Deployment

	Container	Pod	Deployment
What it is	Running process in isolation	Wrapper over container(s)	Manages pods via ReplicaSet
Auto healing	No	No	Yes
Scaling	No	No	Yes
Used in production	Rarely alone	As base unit	Always

Service

A **Service** solves three problems that exist when using pods/deployments alone.

Problem 1 – Pods get new IPs when restarted

When a pod crashes and restarts (auto healing), its IP changes. If other teams or apps are using that IP directly, they will break.

Problem 2 – Load balancing across replicas

With 3 replicas of a pod, you need something to distribute traffic.

Problem 3 – Exposing apps outside the cluster

Pods are only accessible inside the cluster by default.

How Service solves these – Labels and Selectors

Services don't track pod IPs. Instead, they track pods by **labels**. Labels are set on pods and never change (ReplicaSet always creates pods with the same label). So even if a pod restarts with a new IP, the service finds it via label.

Service Types

Type	Who can access
ClusterIP (default)	Only within the cluster
NodePort	Anyone with access to the worker node IP
LoadBalancer	Anyone on the internet (cloud only, uses CCM to create external IP)

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
  type: ClusterIP
```

Ingress

Services of type LoadBalancer create one external IP per service. If you have 100 services, you'd have 100 load balancers — very expensive.

Ingress solves this by using a single load balancer and routing traffic to the right service based on rules (host-based or path-based routing).

How it works

1. Install an **Ingress Controller** (e.g., nginx ingress controller) on your cluster
2. Create **Ingress resources** (YAML files) that define routing rules
3. The ingress controller watches ingress resources and configures the load balancer

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
```

```
name: my-ingress
spec:
  rules:
  - host: myapp.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: my-service
            port:
              number: 80
```

Path-based routing

Send `/payments` traffic to payments service, `/orders` to orders service — all under one domain and one IP.

TLS with Ingress

Store TLS certificates in a Kubernetes Secret, then reference in Ingress:

```
tls:
- hosts:
  - myapp.example.com
  secretName: my-tls-secret
```

TLS Modes

- **SSL Passthrough** – encrypted end to end, load balancer just passes traffic. Secure but can't do path routing or WAF.
- **SSL Offloading (Edge)** – load balancer decrypts, sends plain HTTP to app. Fast but less secure.
- **SSL Bridging (Re-encrypt)** – load balancer decrypts, re-encrypts before sending to app. Most balanced option.

RBAC (Role-Based Access Control)

Controls who (users/services) can do what on your cluster.

Two categories

1. **User management** – human users (devops engineers, developers, QE)
2. **Service accounts** – pods/applications running in the cluster

Key principle

Kubernetes does NOT manage users itself. It delegates user management to external **Identity Providers** (IAM for EKS, LDAP, Okta, Keycloak, etc.)

Service accounts are created directly in Kubernetes as YAML resources.

How RBAC works

Role → defines what actions are allowed (get, list, create, delete) on which resources (pods, secrets, configmaps)

RoleBinding → attaches a Role to a user or service account

For cluster-wide permissions: use **ClusterRole** and **ClusterRoleBinding**

```
ServiceAccount/User + Role → RoleBinding → Permissions granted
```

Why it matters

- Developers should not be able to delete cluster resources
- Secrets should only be accessible to specific roles (principle of least privilege)
- Without proper RBAC, a malicious or buggy pod could damage your cluster

ConfigMaps and Secrets

Problem they solve

Applications need configuration (database URL, port, credentials). These should not be hardcoded in the application or Docker image.

ConfigMap

For **non-sensitive** data. Stored in etcd as plain text.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
```

```
DB_PORT: "3306"
DB_HOST: "mysql-service"
```

Secret

For **sensitive** data (passwords, tokens, certificates). Stored encrypted at rest in etcd.

```
kubectl create secret generic my-secret \
  --from-literal=DB_PASSWORD=mypassword
```

By default Kubernetes uses base64 encoding — not strong encryption. For production, use:

- HashiCorp Vault
- Sealed Secrets
- AWS Secrets Manager

Always restrict access to Secrets using RBAC.

Using ConfigMap in a Pod

As environment variables:

```
env:
- name: DB_PORT
  valueFrom:
    configMapKeyRef:
      name: app-config
      key: DB_PORT
```

As a mounted file (recommended — updates without pod restart):

```
volumes:
- name: config-volume
  configMap:
    name: app-config
volumeMounts:
- name: config-volume
  mountPath: /opt
```

With volume mounts, if you update the ConfigMap, the file inside the pod updates automatically within a few seconds — no pod restart needed. This does NOT work with

environment variables.

Deployment Strategies

1. Rolling Update (default)

Kubernetes replaces pods incrementally. Old pods are removed as new ones come up.

- Default: 25% MaxUnavailable, 25% MaxSurge
- Near-zero downtime
- Always configure **Readiness Probes** so Kubernetes knows when the new pod is ready

bash

```
kubectrl set image deployment/nginx nginx=nginx:1.15
```

Limitation: Once rollout starts, you have limited control. All replicas will eventually be replaced.

2. Canary Deployment

Deploy new version alongside old version. Route a small % of traffic to the new version using Ingress annotations.

```
# On the new version's Ingress:
annotations:
  nginx.ingress.kubernetes.io/canary: "true"
  nginx.ingress.kubernetes.io/canary-weight: "10"
```

- 10% of traffic goes to new version, 90% to old
- Test in production with real data and real users
- Gradually increase the percentage (10 → 30 → 50 → 100)
- Easy rollback: just change weight back to 0

Best for: apps where you cannot afford bad user experience, need production validation.

3. Blue-Green Deployment

Run two identical environments (Blue = current, Green = new).

- Deploy new version fully in "Green"
- Switch load balancer (Ingress) from Blue service to Green service instantly
- Instant rollback: switch back to Blue

Downside: Doubles your resource cost during the transition period. Best for banking/financial services where rollback speed matters more than cost.

Real-time Challenges in Production

Challenge 1 – Resource Sharing

Problem: One team's pod leaks memory and consumes all cluster resources, crashing pods in other namespaces.

Fix:

1. Set **ResourceQuota** per namespace — limits total CPU/memory for all pods in that namespace
2. Set **resource limits** per pod — limits how much one pod can consume

```
resources:
  requests:
    memory: "64Mi"
    cpu: "250m"
  limits:
    memory: "128Mi"
    cpu: "500m"
```

This reduces blast radius: cluster-wide → namespace-wide → single pod.

Challenge 2 – OOMKilled

Problem: Pod goes into CrashLoopBackOff with OOMKilled error.

Troubleshooting:

```
kubectl get pods          # status shows CrashLoopBackOff
kubectl describe pod <name> # shows OOMKilled
```

Fix:

- Take thread dump and heap dump from the pod (for Java: `kill -3`, `jstack`)
- Share with development team
- Developer fixes the memory leak and releases new version
- Deploy new version

Challenge 3 – Cluster Upgrades

Process:

1. Read the release notes for the new Kubernetes version
2. Take etcd backup
3. For each worker node:
 - Drain the node (pods move to other nodes)
 - Cordon the node (make unschedulable)
 - Upgrade kubelet and packages
 - Bring node back, uncordon it
4. Upgrade control plane components (API server, scheduler, etc.)

Do one node at a time — never upgrade all nodes simultaneously in production.

StatefulSets and Persistent Storage

StatefulSet vs Deployment

	Deployment	StatefulSet
Pod names	Random hash	Stable: web-0, web-1, web-2
Startup order	Parallel	Sequential (web-0 must be ready before web-1 starts)
Use case	Stateless apps	Databases, any app needing stable identity

Persistent Volume Workflow

StatefulSet → PersistentVolumeClaim (PVC) → StorageClass → Provisioner → PersistentVolume (PV)

Common issue: StorageClass names differ across platforms.

Platform	Default StorageClass
Minikube	standard
EKS	gp2 (EBS)
AKS	default (Azure Disk)
GKE	standard (GCE PD)

If your StatefulSet works on EKS but not on Minikube, check the `storageClassName` in the PVC template.

```
kubectl get storageclass          # see available classes on your cluster
kubectl get pvc                  # check PVC status
kubectl describe pod <name>      # if pending, check for PVC binding errors
```

Note: Deleting a StatefulSet does NOT delete its PVCs. You must delete them manually:

bash

```
kubectl delete pvc <pvc-name>
```

CSI Driver

For external storage services (NetApp, Portworx, etc.) not natively supported by your cloud:

1. Install the vendor's CSI driver (usually via Helm)
2. Create a StorageClass pointing to that driver
3. Reference the StorageClass in your PVC

Essential kubectl Commands

```
# Cluster info
kubectl get nodes
kubectl get nodes -o wide
kubectl cluster-info

# Pods
kubectl get pods
kubectl get pods -o wide
kubectl get pods -w          # watch live
kubectl describe pod <name>
kubectl logs <pod-name>
kubectl logs <pod-name> --previous
kubectl exec -it <pod-name> -- /bin/bash
kubectl delete pod <name>

# Deployments
kubectl get deploy
kubectl apply -f deployment.yaml
kubectl delete deploy <name>
```

```
kubectl rollout status deploy/<name>
kubectl rollout undo deploy/<name>

# All resources in namespace
kubectl get all
kubectl get all -A # all namespaces

# Namespaces
kubectl get ns
kubectl get pods -n <namespace>

# Describe and debug
kubectl describe <resource> <name>
kubectl get events

# Config
kubectl config get-contexts
kubectl config use-context <name>

# Labels
kubectl label node <node-name> key=value
kubectl taint nodes <node-name> key=value:NoSchedule
```